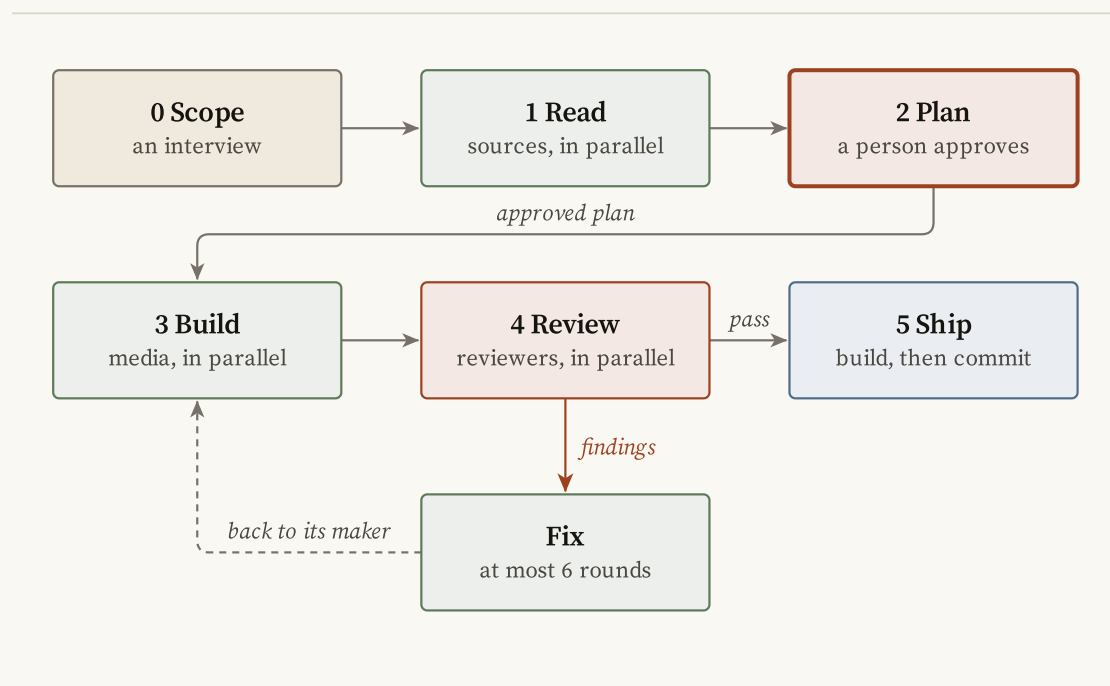


Lessons with a tutor inside

How a team of agents turns course material into a lesson a student can talk to

lesson-builder is a Claude Code skill. Point it at a course's slides, notes and past problems and it builds a lesson: a small web app with the topics in order, the equations typeset, graphs the reader can move, animations, demos, and a tutor beside the text that knows what the page says. It works in six phases. Agents it spawns do most of the work, and a person approves the plan before any of it starts.

FIGURE 1 *six phases, and one of them waits for a person*



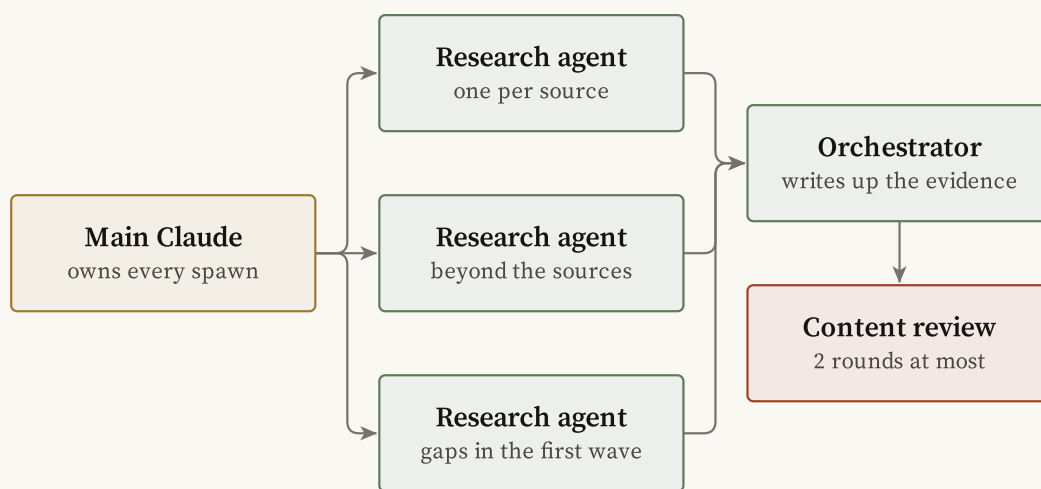
Sage is agents at work, rust a check that can say no, slate what ships. The heavy box is the one place a person decides.

Reading and planning

The first three phases decide what the lesson teaches, and a person signs it off.

Phase 0 is an interview: which materials count, how far past them to go, and where the lesson ships. A build with no one watching answers from the course's notes and lists what it assumed.

FIGURE 2 *phase 1 reads every source at once*



Ochre is the session, sage the agents that work, rust the one that checks them.

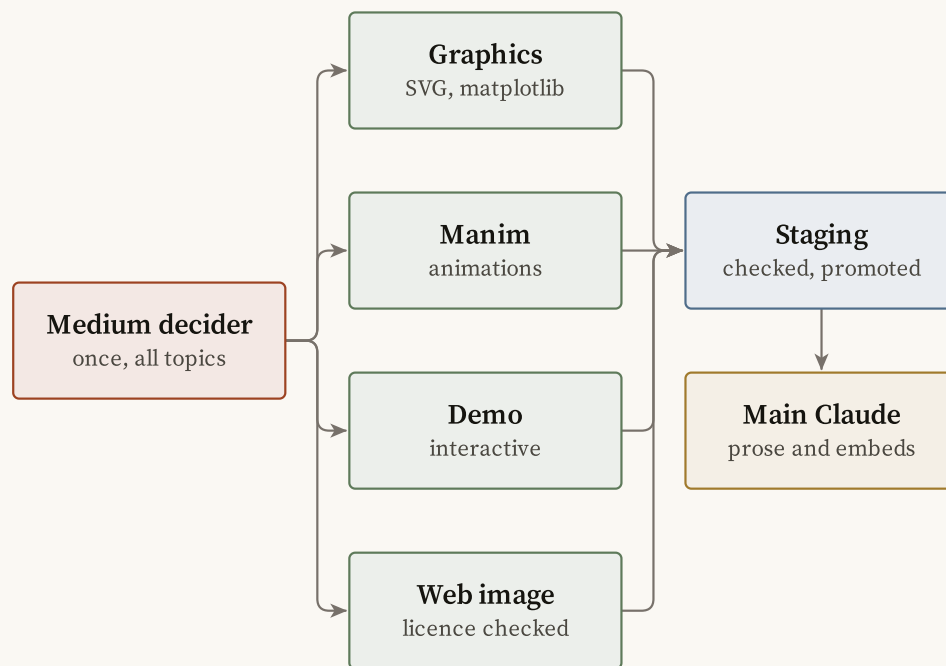
Phase 1 reads. Main Claude spawns one research agent per source, more for ground the sources don't cover, and a second wave for what the first missed. The content orchestrator writes their findings up as one package, and a content review can send it back.

Phase 2 writes the plan. Each topic gets a teaching arc: the question it answers, the steps that get the student there, and what they should then be able to do. One medium decider picks the media for the whole lesson at once, so they stay varied. Then the plan goes to a person, and silence is not a yes. A build with no one watching stops until an instruction quotes the plan's hash, so an approval always names the exact plan somebody read.

Building

Phase 3 builds every piece of media at once, and nothing a specialist makes reaches the lesson until it has been checked.

FIGURE 3 *one decision, then one specialist per item*



Rust is the decider, sage the specialists, slate the staging area, ochre the session.

Main Claude spawns one specialist per item in the plan, all at once, and none of them writes into the lesson. What they make goes into a staging area, and a script moves each file into place only once it has checked this run made it, so a crashed run leaves the old lesson as it was. Main Claude writes the prose and the Desmos and GeoGebra figures itself. An update to an existing lesson runs in a git worktree of its own, so the author's checkout is never touched while it runs.

The agent team

Twelve agents ship with the skill, and the session running the build picks each one's model.

TABLE 1 *the twelve agents and where they work*

AGENT	PHASE	WHAT IT DOES
Research	1	Pulls facts out of one source, or researches a topic, with sources
Content orchestrator	1	Writes the workers' evidence up as one package
Content review	1, 4	Checks accuracy, then how each explanation unfolds
Medium decider	2	Picks the media for every topic, in one pass
Graphics	3	SVG graphs and matplotlib figures
Manim	3	Short animations, where motion carries the point
Interactive demo	3	Demos a student can drive, from shared parts
Web image	2, 3	Real photographs and spectra, licence checked
Code review	4	The lesson file: imports, maths, that it parses
Visual QA	4	One figure against a full presentation rubric
Scientific accuracy	4	Whether a figure shows what the science says
Interaction	4	Drives a running demo and tries every control

Main Claude spawns every agent itself, because a subagent can't spawn another. Each one writes its full output to a file and hands back a summary, so the session stays small and the detail is there for whoever needs it.

The agents that judge what the lesson teaches and whether it's right run on the session's own model. The ones that make media or apply a rubric default to a smaller one. Main Claude can raise either, and nothing is lowered to save money unless the person asking said so.

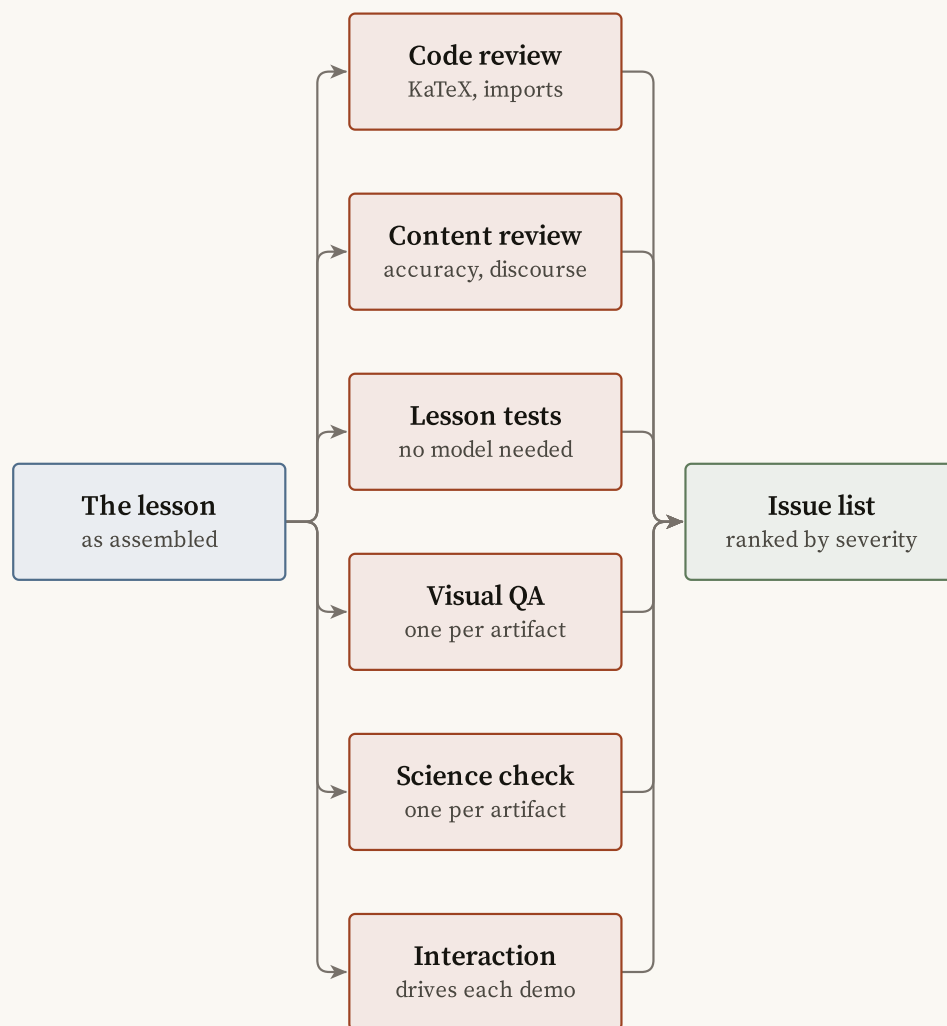
Table 1: SKILL.md, *Agent team*, and the description line of each file in *agents/*. The models: SKILL.md, *Model policy* and *Quality policy*, and the model: line of each agent. Who spawns what: *Agent team*.

Review, and the fix loop

Every reviewer runs at once, and none of them can edit.

Phase 4 starts every review at once. Each figure gets two reviewers, one for how it looks and one for whether the science is right, because they miss different things.

FIGURE 4 *phase 4 reviews everything in one batch*

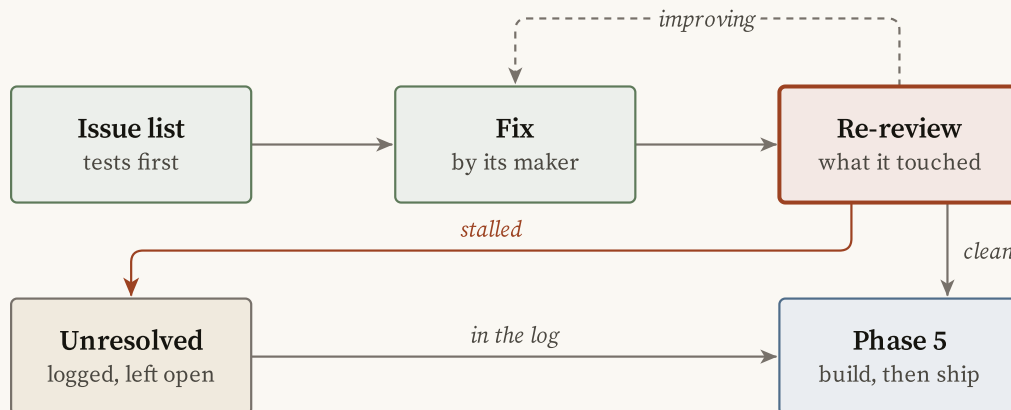


Slate is the lesson, rust the six checks that can say no, sage the list they feed.

Every finding goes into one list, written to the run record before anything is fixed. A reviewer never fixes what it found, so no model grades its own work. A figure goes back

to the specialist that made it, and main Claude fixes the code and prose. Failures a test can prove, like a parse error or a broken build, go first, because the rest are often their symptoms.

FIGURE 5 *the loop that follows*



Sage is the work, rust the check the loop turns on, and slate the next phase. The dashed line is another round; the rust line is a thread given up on.

The loop stops early rather than churn, because a loop that keeps rewriting breaks work that was already right. A thread is given up when two rounds in a row don't shrink its issues, and it is listed as unresolved when the lesson ships. A finding that shows the plan itself was wrong stops the loop and goes back to a person.

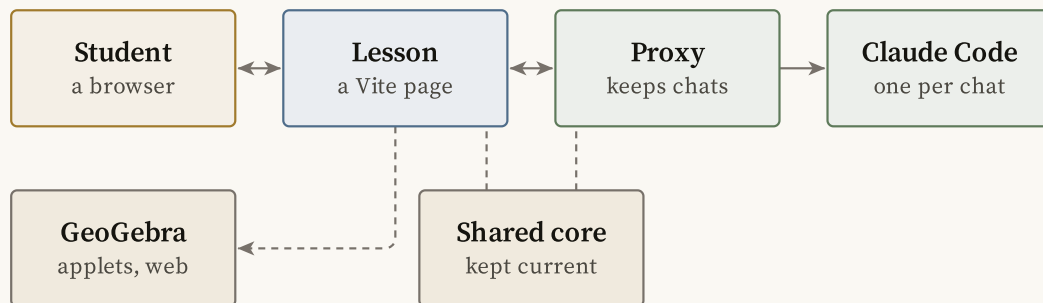
Phase 5 builds the lesson once more, keeps course material out of the commit, and pushes. If the plan asked for a printed companion, it builds that too and links it from the lesson.

Figure 4: `references/phase-4-review.md`, *Parallel reviews*. Figure 5, who fixes what and the stop rules: `references/phase-4-review.md`, *Progress-aware fix loop* and *Stop rules*. Phase 5: `SKILL.md`, *Pipeline*, and `references/phase-5-deploy.md`.

What a lesson is made of

A handful of files of its own, and a shared core that does the rest.

FIGURE 6 *what runs when a student opens a lesson*



Ochre is the person, slate the page they read, sage the two processes behind it, and grey what the page loads but does not own: the code every lesson shares, and GeoGebra's applets. Dashed lines are imports, not traffic.

Each lesson is a Vite and React project in a folder under its course. The one long file is the lesson's own text and figures. The page shell, the tutor chat, the maths and the proxy come from a shared core that every lesson imports, so a fix to the chat reaches every lesson at once. Each build first brings the workspace's copy of the core up to date with the skill, and puts the old one back if the new one fails its smoke test.

At a desk the page has a contents rail, a column of text and a tutor that docks at the side or opens in its own window. On a phone the contents and the tutor slide up as sheets and a wide equation scrolls sideways instead of shrinking. If KaTeX never loads, the reader sees the LaTeX source rather than a blank. A topic gets at most two GeoGebra applets, because a phone pays for every one.

The layout and the shared core: `SKILL.md` and `references/bootstrap/_lesson-core/`. The proxy's routes: `_lesson-core/server/proxy.js`. The shell: `_lesson-core/ui/LessonShell.jsx`. The refresh: `scripts/core-refresh.sh`. GeoGebra: `references/geogebra.md`.

TABLE 2 *the files a lesson keeps in its own folder*

FILE	WHAT IT DOES
src/<slug>.jsx	The lesson itself: its topics, its prose, and what the tutor is told about each topic
src/main.jsx	Mounts it
index.html	The page it mounts into
package.json	Its own dependencies: React, Vite, KaTeX
server/proxy.js	One line that starts the shared proxy for this lesson
vite.config.js	Points the @core import at the shared core
test_lesson.cjs	The checks the lesson must pass before it ships
CLAUDE.md	What an agent editing this lesson needs to know
.gitignore	Keeps course material private by default
lesson_build.log.md	The build log, rendered from the run record

The tutor can do more than answer. Through tagged messages it can change a graph the student is looking at, draw a demo, or propose an addition to the lesson that the student can accept. A tag it only quotes, in a code block or a quoted line, does nothing, and whatever it writes is rendered through an allowlist, never as raw HTML. A student can Ctrl-click any part of the page to bring it into the chat, or open a side thread on a paragraph that folds back into the main chat as a summary.

Chats outlast the tab. Closing one keeps the chat, reopening it restores the history, and a message sent while the tutor is still replying waits its turn. If the same chat is open in two tabs, one holds it and the other can take it over.

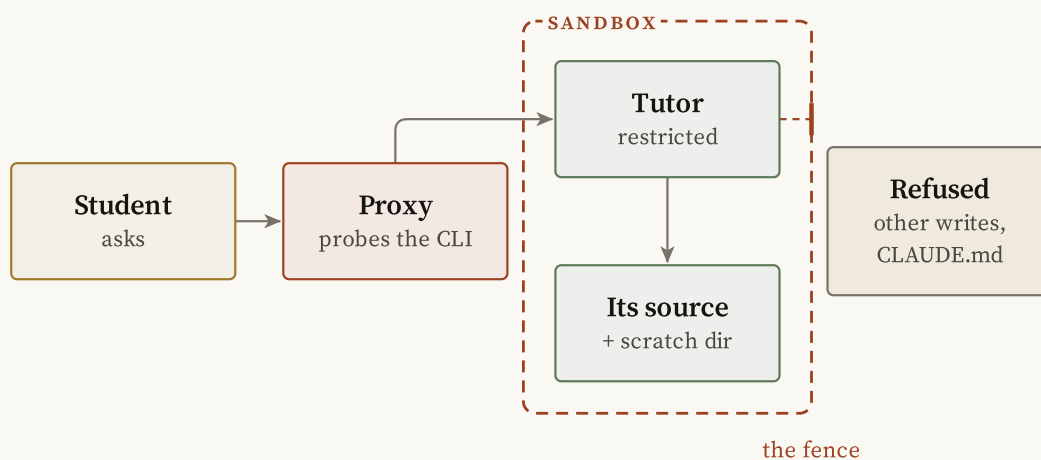
Table 2: SKILL.md, the per-lesson file list. The tags and renderer: _lesson-core/chat/processResponse.js and safeRender.js. Chats that persist: _lesson-core/chat/tutorApi.js.

A tutor that stays in its lesson

The tutor is a real Claude Code session, so the proxy decides what it can reach.

Each chat is its own Claude Code process, started by the lesson's proxy, so the tutor has real tools. It can read the course files, search the web, run commands and call on agents of its own.

FIGURE 7 *where the tutor may write*



Ochre is the student, rust the check that refuses, sage the tutor and what it may write, and grey everything else. The rust dashed line is the fence.

Every tutor starts restricted. It loads no CLAUDE.md, hooks or user MCP servers, so it carries no one's instructions but the lesson's. It may write only the lesson's source and a scratch folder, because the rest of the lesson folder holds files that run outside any sandbox. Its shell commands run in a sandbox with the same limits, and anything else is denied without a prompt. Before the proxy starts a tutor, it checks that the CLI offers every one of those options. If one is missing, no tutor starts. If the check only timed out on a busy machine, the proxy tries again rather than giving up until a restart.

The tutor has a team of its own: one agent that notices when a chat has filled a gap in the lesson, one that knows what the lesson already says, and copies of the pipeline's agents for a graph, a check or a quick piece of research while the student waits.

The options, the probe and the refusal: `_lesson-core/server/proxy.js`, *Tutor confinement*, and its fixture `tests/tutor-confinement/`. The tutor's team: `SKILL.md`, *Runtime tutor team*.

How it is checked

Two layers of tests, and nothing that can block a merge needs a model or a browser.

The first layer is inside each lesson. Its checks parse the file, confirm every topic has what the tutor needs, and build the tutor's whole prompt to see that it fits on the CLI's command line. A prompt too long for that has to go in on standard input, where it loses its weight as instructions, so the size is a test and not a guideline.

The second layer is the skill's own gate, which every pull request must pass to merge. Each of its scenarios builds its own fixture and runs without a model, using a fake claude where it needs the CLI. The runs that need a browser, the real CLI or a model aren't skipped in silence: the gate prints each one with the command that runs it.

A person decides what a lesson teaches. Everything after that has something that can say no.

THE ONE CLAIM THIS DOCUMENT MAKES

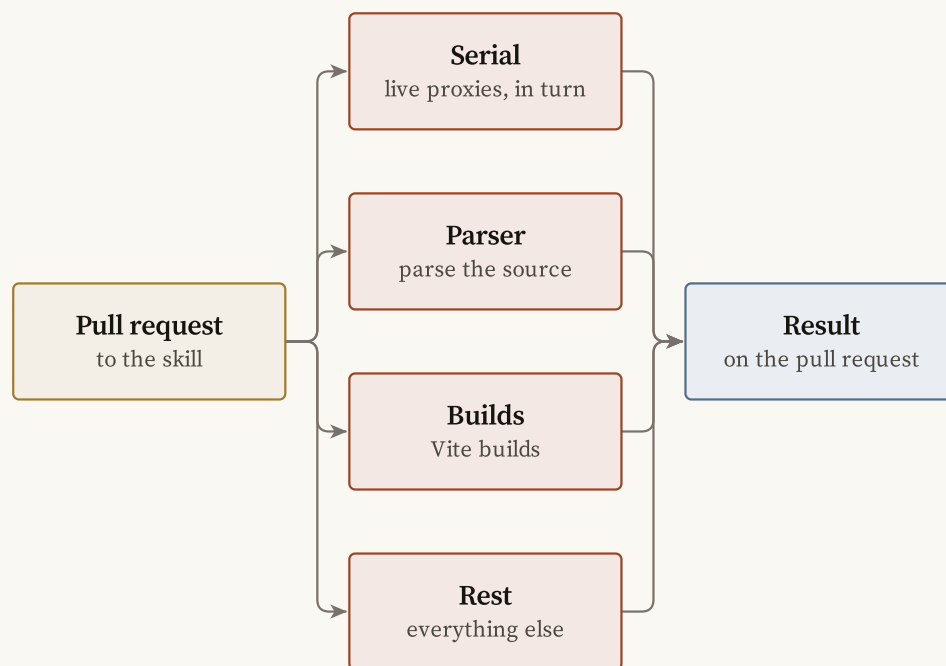
WHAT THE GATE DOES NOT JUDGE

Whether a lesson teaches well is judged by a reviewing model against a rubric. The evals that test that reviewer and the tutor on blind cases need a model too, so they are not in the gate, and a lesson can pass every test and still teach badly.

The lesson's checks: `references/checklists.md`. The gate, its fixtures and its exclusions: `tests/check.sh`. The prompt ceiling: `_lesson-core/constants/promptBudget.js`. Teaching evals: `evals/teaching/README.md`.

The gate also runs on GitHub Actions for every pull request, split into four jobs. The scenarios that start real proxies run one at a time, the parser and build scenarios get jobs of their own, and the last job runs whatever is left, read from the gate itself, so a new scenario is covered without anyone editing the workflow. It needs no secrets and deploys nothing.

FIGURE 8 *the same gate, on GitHub Actions*



Ochre is the change, rust the four jobs that can each say no, slate where the result shows.

In use

Most of what it has learned came from real lessons going wrong.

It builds the lessons for a set of university courses. A built copy is published behind an access list, with the tutor live, and a lesson can have a printed companion made by the skill that made this document.

TABLE 3 *what went wrong, and what changed*

WHAT HAPPENED	WHAT CHANGED
Cancelling a turn could kill the proxy itself	A process group is signalled only while the CLI still leads it
A dash in a long reply came out as a replacement character	The reader carries a character across a pipe-buffer boundary
A lesson built to the documented budget shipped a demoted prompt	The size is measured from the real prompt, and tested
A long reply was lost when the tab's stream dropped	The chat re-attaches to the turn still running
One wrongly declared graph setting stopped the tutor in several lessons	The chat skips the setting, and a new check fails the lesson
A busy machine timed out the CLI check, and the tutor stayed off until a restart	A timed-out check is retried, not taken as a no
A tag the tutor only quoted still changed the page	Tags inside code and quotes are inert

Each fix merged with a scenario in the gate that covers it. What's still open is structural: one app shell that mounts every lesson, typed actions in place of the tagged messages, and a tutor for the author kept apart from the student's.